

Laboratorium nr 10

Protokoły dowodzenia z wiedzą zerową — interaktywny protokół Schnorra

Liczba punktów: Zadanie: 2pkt (+2pkt). Pytania kontrolne: 1pkt. Razem: 5pkt.

Forma zaliczenia: live demo + pytania kontrolne

1. Opis laboratorium

Celem laboratorium jest poznanie i implementacja interaktywnego protokołu dowodzenia z wiedzą zerową (*zero-knowledge proof of knowledge*, ZKPoK) na przykładzie protokołu identyfikacji Schnorra (Schnorr 1989). W ramach laboratorium:

1. Zaimplementujesz interaktywny protokół Schnorra w grupie cyklicznej rzędu pierwszego $\mathbb{Z}_p^*$;
2. Podepniesz go jako klient do serwera autoryzacji udostępnianego przez prowadzącego i wykonasz uwierzytelnienie bez przysyłania hasła.

1.1. Język protokołów ZKP

Protokół dowodzenia z wiedzą zerową jest dwustronnym protokołem między **Proverem** P a **Verifierem** V . Centralnym pojęciem jest **relacja** R określająca, co znaczy “dowód poprawny”:

- x — *statement* (wypowiedź publiczna, wejście jawne);
- w — *witness* (świadek, sekret znany tylko Proverowi);
- $R(x, w) \in \{0, 1\}$ — funkcja relacji; Prover twierdzi: “znam w takie, że $R(x, w) = 1$ ”;
- $b \in \{0, 1\}$ — *decision bit* wyjściowy Verifiera (1 = accept, 0 = reject).

Dla protokołu Schnorra relacja jest następująca:

$$R_{\text{DLP}} = \{((g, p, q, y), x) : y \equiv g^x \pmod{p}\}.$$

Statement to czwórka (g, p, q, y) , a witness to logarytm dyskretny x . Prover dowodzi znajomości x bez ujawniania jego wartości.

1.2. Trzy własności bezpieczeństwa

Protokół ZKP musi spełniać trzy własności, z których każda chroni przed innym typem ataku i ma odrębną definicję formalną.

(1) Completeness (zupełność). Jeśli $R(x, w) = 1$ i obie strony postępują zgodnie z protokołem, to V wyprowadza $b = 1$ z prawdopodobieństwem 1.

(2) Knowledge Soundness (KSND, niezawodność wiedzy). Chroni przed nieuczciwym Proverem P^* . Jeśli P^* przekonuje V z istotnym prawdopodobieństwem, to **istnieje algorytm Knowledge Extractor** $\mathcal{E}$, który mając dostęp do P^* potrafi wydobyć poprawnego świadka w^* . Operacyjnie: dwa akceptowane transkrypty (a, e_1, z_1) i (a, e_2, z_2) z różnymi wyzwaniem $e_1 \neq e_2$ pozwalają wyliczyć w . To dowodzi, że P^* *musiał* znać sekret.

(3) Zero-Knowledge (ZK). Chroni przed nieuczciwym Verifierem V^* . **Istnieje algorytm Simulator** $\mathcal{S}$, który mając tylko statement x (bez witness w) produkuje transkrypty *nieodróżnialne* od transkryptów prawdziwej interakcji. To dowodzi, że V^* nie uzyskuje żadnej informacji o w , której nie mógłby uzyskać sam, znając tylko x .

Uwaga. Klasyczna definicja ZK zakłada uczciwego Verifiera (HVZK — *Honest Verifier Zero-Knowledge*). Pełne ZK wobec dowolnego V^* wymaga dodatkowych technik (rewinding, Fischlin transform). W tym laboratorium pracujemy z HVZK.

1.3. Σ -protokoły

Protokół Schnorra należy do rodziny Σ -**protokołów**: trzy-wiadomościowych protokołów *publicznych monet* (public-coin), w których wyzwanie V jest losowane jawnie. Schemat:

Krok 1 (commitment): $P \rightarrow V : a$
Krok 2 (challenge): $V \rightarrow P : e$
Krok 3 (response): $P \rightarrow V : z$

Verifier akceptuje **wtedy i tylko wtedy**, gdy $\varphi(x, a, e, z) = 1$ dla pewnego predykatu φ .

1.4. Protokół identyfikacji Schnorra

Parametry publiczne: liczba pierwsza p , liczba pierwsza $q \mid p - 1$, generator g podgrupy rzędu q w $\mathbb{Z}_p^*$.
Zalecane: $|p| \geq 2048$ b. W laboratorium używamy parametrów RFC 7919 ffdhe2048 (sekcja 5.1).

Statement i witness: $x_{\text{pub}} = (g, p, q, y)$; witness $w = x_{\text{priv}}$, takie że $y = g^{x_{\text{priv}}} \bmod p$.

Krok	Strona	Operacja
1. Commitment	P	losuje $r \in_R \{1, \dots, q - 1\}$; oblicza $a = g^r \bmod p$; wysyła a
2. Challenge	V	losuje $e \in_R \{0, \dots, 2^t - 1\}$; wysyła e
3. Response	P	oblicza $z = (r + e \cdot x_{\text{priv}}) \bmod q$; wysyła z
4. Verify	V	akceptuje wtw. $g^z \equiv a \cdot y^e \pmod{p}$

Completeness:

$$g^z = g^{r+e \cdot x_{\text{priv}}} = g^r \cdot (g^{x_{\text{priv}}})^e = a \cdot y^e \pmod{p}.$$

Knowledge Extractor (intuicja). Z dwóch akceptowanych transkryptów (a, e_1, z_1) i (a, e_2, z_2) z różnymi wyzwaniami $e_1 \neq e_2$ wynika:

$$z_1 - z_2 \equiv (e_1 - e_2) \cdot x_{\text{priv}} \pmod{q},$$

skąd:

$$x_{\text{priv}} = (z_1 - z_2) \cdot (e_1 - e_2)^{-1} \bmod q.$$

Konstrukcja Extractora dowodzi, że Prover przekonujący Verifiera dwukrotnie *musi* znać x_{priv} . Ta sama algebra leży u podstaw ataku na podpisy Schnorra / ECDSA z powtórzonym nonce (między innymi incydent Sony PlayStation 3, 2010) — temat omawiany na wykładzie nr 10.

1.5. Zastosowania w systemach produkcyjnych

Protokół zaimplementowany w niniejszym laboratorium jest dydaktycznym przedstawieniem matematyki, która stanowi rdzeń następujących systemów używanych komercyjnie:

- **Ed25519** (RFC 8032, NIST FIPS 186-5) — wariant Schnorra na krzywej eliptycznej Edwards Ed25519. Standard SSH, TLS 1.3, Signal, WhatsApp, WireGuard, Apple Passkeys, Android Keystore.
- **Bitcoin Taproot** (BIP-340, aktywne od 2021 r.) — Schnorr na krzywej secp256k1, używany do każdej nowej transakcji Bitcoin.
- **FIDO2 / WebAuthn** — globalny standard uwierzytelnienia bez haseł (Google, Apple, Microsoft); pod spodem ECDSA lub Ed25519.

- **Konsensus blockchain** — Solana, Polkadot, Cardano używają wariantów Schnorra do podpisywania transakcji w sieciach o przepustowości rzędu 10^4 – 10^5 tps. W każdym z tych systemów występują dokładnie te same trzy kroki: commitment, challenge, response. Różni je **grupa**, w której działają, oraz **sposób generowania wyzwania** e . W zapisie multiplikatywnym (jak w niniejszym laboratorium) weryfikacja ma postać:

$$g^z \equiv a \cdot y^e \pmod{p}.$$

W zapisie addytywnym dla krzywych eliptycznych identyczna zależność wygląda następująco:

$$z \cdot G = R + e \cdot Y,$$

gdzie G jest punktem bazowym krzywej, $Y = x_{\text{priv}} \cdot G$ jest kluczem publicznym, $R = r \cdot G$ jest commitment'em, a operacje $+$ i $\cdot$ to odpowiednio dodawanie punktów oraz mnożenie punktu przez skalar. Algebraicznie jest to ta sama tożsamość, co weryfikacja z sekcji 1.4 — różni się tylko notacją podyktowaną wyborem grupy.

Sposób uzyskiwania wyzwania w wersji nieinteraktywnej (podpis cyfrowy) wykorzystuje transformację Fiat-Shamir:

$$e = H(\text{pk} \| a \| m),$$

gdzie H jest kryptograficzną funkcją skrótu (SHA-256, SHA-512 lub w przypadku Ed25519 — SHA-512 z dodatkową strukturą). Pominięcie pk , a lub m w wejściu H otwiera atak; temat jest omawiany na wykładzie nr 10.

2. Materiały

1. C.-P. Schnorr, *Efficient Identification and Signatures for Smart Cards*, CRYPTO '89, https://link.springer.com/chapter/10.1007/0-387-34805-0_22
 2. A. Menezes, P. van Oorschot, S. Vanstone, *Handbook of Applied Cryptography*, rozdz. 10, <http://cacr.uwaterloo.ca/hac/about/chap10.pdf>
 3. D. Boneh, V. Shoup, *A Graduate Course in Applied Cryptography*, rozdz. 19, <https://toc.cryptobook.us/>
 4. A. Fiat, A. Shamir, *How to Prove Yourself*, CRYPTO '86, https://link.springer.com/chapter/10.1007/3-540-47721-7_12
 5. S. Goldwasser, S. Micali, C. Rackoff, *The Knowledge Complexity of Interactive Proof-Systems*, STOC '85, <https://doi.org/10.1145/22145.22178>
 6. T. Silde, A. Takahashi, *Zero Knowledge Proofs: Challenges, Applications, and Real-world Deployment*, NIST Workshop on Privacy-Enhancing Cryptography 2024, <https://csrc.nist.gov/Events/2024/wpec2024>
 7. D. Pointcheval, J. Stern, *Security Arguments for Digital Signatures and Blind Signatures*, Journal of Cryptology 2000, <https://link.springer.com/article/10.1007/s001450010003>
 8. M. Green, *Zero Knowledge Proofs: An Illustrated Primer*, <https://blog.cryptographyengineering.com/2014/11/27/zero-knowledge-proofs-illustrated-primer/>
 9. BIP-340: *Schnorr Signatures for secp256k1*, <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki>
 10. RFC 7919: *Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security*, <https://www.rfc-editor.org/rfc/rfc7919.html>
 11. RFC 8032: *Edwards-Curve Digital Signature Algorithm (EdDSA)*, <https://www.rfc-editor.org/rfc/rfc8032.html>
-

3. Zadanie do wykonania

Napisz aplikację realizującą poniższe zadanie. Język programowania **dowolny**. Wymagana jest obsługa arytmetyki dużych liczb.

Klient referencyjny w języku Python (`client.py`) zawiera gotowy szkielet HTTP, parametry grupy oraz puste funkcje kryptograficzne (oznaczone `TODO STUDENT`) do uzupełnienia. Plik dostępny jest na zajęciach.

3.1. Zadanie 1 (2 pkt + 2 pkt) — implementacja interaktywnego protokołu Schnorra i klienta uwierzytelniającego

Zaimplementuj protokół identyfikacji Schnorra zgodnie z opisem w sekcji 1.4 oraz podepnij go jako klienta do serwera pokazowego.

Część A: lokalna implementacja.

1. Pobierz parametry (p, q, g) z serwera (GET `/params`). Zweryfikuj lokalnie:
 - $g^q \equiv 1 \pmod{p}$ (generator ma rząd dzielący q);
 - $g \neq 1$.
2. Zaimplementuj funkcję `keygen()` zwracającą parę (x_{priv}, y) , gdzie $x_{\text{priv}} \in_R \{1, \dots, q-1\}$ oraz $y = g^{x_{\text{priv}}} \pmod{p}$.
3. Zaimplementuj cztery rozdzielne funkcje protokołu:
 - `prover_commit()` $\rightarrow (r, a)$;
 - `verifier_challenge(t_bits)` $\rightarrow e$;
 - `prover_response(r, e, x_priv)` $\rightarrow z$;
 - `verifier_check(y, a, e, z)` $\rightarrow \text{bool}$.
4. Wykonaj testy lokalne:
 - **test completeness:** 100 przebiegów z poprawnym kluczem — wszystkie akceptowane;
 - **test odporności:** mutacja z lub a o 1 $\rightarrow$ odrzucenie;
 - **test z fałszywym kluczem:** Prover używa $x' \neq x_{\text{priv}} \rightarrow$ odrzucenie.

Część B: klient uwierzytelniający.

1. Zaimplementuj `register(username, y)` — wywołanie POST `/v1/register`.
2. Zaimplementuj `login(username, x_priv)` jako pełną sekwencję:
 - obliczenie (r, a) lokalnie przez `prover_commit()`;
 - POST `/v1/login/start {username, a}` — otrzymanie wyzwania e oraz `session_id`;
 - obliczenie $z = (r + e \cdot x_{\text{priv}}) \pmod{q}$ lokalnie przez `prover_response`;
 - POST `/v1/login/finish {session_id, z}` — otrzymanie tokenu sesji (status 200) lub odrzucenia (401).

Demonstracja do oceny. Live demo przed prowadzącym składa się z trzech kroków wykonanych na żywo na uruchomionym kliencie:

1. Rejestracja konta studenta (np. `nazwisko@lab`) — odpowiedź 200;
2. Udane uwierzytelnienie — odpowiedź 200 z tokenem sesji;
3. Próba uwierzytelnienia z podstawioną fałszywą wartością z (np. zwiększoną o 1) — odpowiedź 401.

4. Pytania kontrolne (1 pkt)

1. Dlaczego wyzwanie e musi być losowane *po* wysłaniu commitment a ? Jakie konsekwencje miałyby odwrócenie kolejności?
 2. Klucz prywatny x_{priv} nigdy nie opuszcza klienta — serwer otrzymuje tylko y , a i z . Jaką to ma przewagę nad uwierzytelnieniem hasłem w sytuacji, gdy serwer zostaje skompromitowany (np. wyciekła baza danych)?
-

5. Wskazówki implementacyjne

5.1. Parametry grupy (RFC 7919 ffdhe2048)

Identyczne wartości używa serwer pokazowy. Pełna postać liczbową w pliku `client.py`. Schematycznie:

$$\begin{aligned} p &= \text{FFFFFFFF ... FFFFFFFFF} \quad (2048 \text{ bitów}) \\ q &= (p - 1)/2 \quad (2047 \text{ bitów, } \textit{safe prime}) \\ g &= 2 \end{aligned}$$

Wymagana lokalna weryfikacja: $g^q \equiv 1 \pmod{p}$ oraz $g \neq 1$.

5.2. Serializacja liczb przy komunikacji z serwerem

Wszystkie wartości liczbowe przesyłane są w formacie **lowercase hex string** (bez prefiksu `0x`). Konwencja jednolita dla obu kierunków komunikacji. Format JSON Number nie jest stosowany z uwagi na 53-bitowe ograniczenie precyzji.

5.3. Losowanie liczb

Do losowania liczb stosuj wyłącznie kryptograficznie bezpieczne generatory:

- Python ≥ 3.8 : moduł `secrets`, funkcja `secrets.randbelow(q - 1) + 1`;
- Java: `java.security.SecureRandom`;
- C/C++: `/dev/urandom`, biblioteka GMP z dobrym źródłem entropii;
- Rust: `crate rand` z `OsRng`.

Nie należy używać generatorów pseudolosowych przeznaczonych do zastosowań niekryptograficznych (`random.randint` w Pythonie, `java.util.Random`, `rand()` w C).

5.4. Arytmetyka modularna

Najczęstszym źródłem błędów w implementacji protokołu jest pomylenie modulo. Obowiązuje zasada:

- bazę redukujemy mod p (operacje na elementach grupy $\mathbb{Z}_p^*$);
- wykładniki redukujemy mod q (operacje w pierścieniu $\mathbb{Z}_q$, rząd podgrupy).

W szczególności: $z = (r + e \cdot x_{\text{priv}}) \bmod q$, **nie** mod p .

5.5. Długość wyzwania

W laboratorium przyjmujemy $t = 256$ bitów, tj. $e \in_R \{0, \dots, 2^{256} - 1\}$. Wartość zgodna z długością wyjścia SHA-256. Błąd *knowledge soundness* wynosi 2^{-256} na sesję. Wartość ta jest stała i nie jest udostępniana przez /params — klient zakłada ją z góry.

5.6. Endpointy serwera pokazowego

Endpoint	Metoda	Cel
/params	GET	parametry grupy (p, q, g)
/v1/register	POST	rejestracja konta z kluczem publicznym y
/v1/login/start	POST	rozpoczęcie uwierzytelnienia (commitment a)
/v1/login/finish	POST	zakończenie uwierzytelnienia (response z)

Adres serwera podaje prowadzący na zajęciach.